

```

class Solution {
public:
    double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
        //求出两个数组的总长度
        int arrLen = nums1.size() + nums2.size();
        if(arrLen == 0)return 0;
        //中位数索引
        int midIndex = arrLen/2;
        //如果是偶数的话 中位数索引是 midIndex 和(midIndex-1)这两个对应的值除以 2
        //如果是奇数的话 中位数索引就是 midIndex 所以至少要找到 midIndex 这个位置和他前一个位置
        //双指针 两个数组两个指针
        int i=0,j=0;
        //新的数组
        vector<int>newArr;
        //i 不能越界 并且 j 不越界 并且新数组的元素没到中位
        while(i < nums1.size() && j < nums2.size() && newArr.size()<=midIndex){
            //谁小谁进 newArr
            if(nums1[i] < nums2[j]){
                newArr.push_back(nums1[i++]);
            }else{
                newArr.push_back(nums2[j++]);
            }
        }
        //让剩余元素 在没到达 midIndex 之前的元素也进入 newArr
        while(i<nums1.size() && newArr.size()<=midIndex){

```

```

        newArr.push_back(nums1[i++]);
    }

    while(j<nums2.size() && newArr.size()<=midIndex){
        newArr.push_back(nums2[j++]);
    }

    //最后根据情况返回
    if(arrLen%2==0){
        return (newArr[midIndex] + newArr[midIndex-1])/2.0;
    }else{
        return newArr[midIndex];
    }
}

};

```

确定目标位置 (Target):

计算两个数组的总长度 arrLen，并找到中位数所在的索引 midIndex = arrLen / 2。

如果总长度是奇数，中位数就是 newArr[midIndex]；如果是偶数，则需要用到 newArr[midIndex] 和 newArr[midIndex - 1]。因此，你只需要通过双指针把合并后的数组推导到 midIndex 这一位即可，不需要把所有元素都合并完。

双指针线性扫描:

分别用 i 和 j 指向 nums1 和 nums2 的开头。

通过 while 循环比较 nums1[i] 和 nums2[j] 的大小，把较小的元素放入 newArr 中，同时控制循环在 newArr.size() <= midIndex 时停下。

处理剩余元素:

如果其中一个数组先遍历完，会通过后续的 while 循环继续把另一个数组的元素填入 newArr，直到填满到需要的 midIndex 位置。

分类返回结果:

根据 arrLen 的奇偶性，如果是偶数，返回 (newArr[midIndex] + newArr[midIndex - 1]) / 2.0；如果是奇数，直接返回 newArr[midIndex]。